

File I/O (파일 입출력)

학습 체크리스트

- ☐ 자바 바이트 스트림과 문자 스트림의 흐름 구조 이해
- ☐ 버퍼링(Buffer)의 필요성 및 Files 유틸리티와 Buffer 기반 읽기/쓰기 구현
- ☐ HttpClient를 활용한 이미지 OpenAPI 조회 및 바이너리 다운로드 습득

자바 스트림(Stream)의 이해

스트림 (Stream)

데이터가 단방향으로 흐르는 통로로, 입출력 장치와 프로그램 사이의 데이터 전송을 일관되게 다루기 위한 논리적 구조

- **단방향 흐름**: 입력 스트림(Input)과 출력 스트림(Output)이 분리되어 데이터가 한 방향으로만 흐름
- **바이트 스트림**: 이미지, 동영상 등 바이너리 데이터를 1바이트 단위로 처리하는 가장 기본적인 통로
- **문자 스트림**: 텍스트 데이터를 2바이트 단위로 처리하며, UTF-8 등의 문자 인코딩을 자동으로 반영함

입출력 스트림의 분류 비교

구분	바이트 스트림 (Byte Stream)	문자 스트림 (Character Stream)
처리 단위	1 바이트 (8-bit)	2 바이트 (16-bit)
대상 데이터	이미지, 오디오, 비디오, 실행 파일 등	순수 텍스트 파일, 마크다운, XML 등
최상위 클래스	<code>InputStream</code> / <code>OutputStream</code>	<code>Reader</code> / <code>Writer</code>
대표 구현체	<code>FileInputStream</code> , <code>BufferedInputStream</code>	<code>FileReader</code> , <code>BufferedReader</code>

텍스트 버퍼링 기반 입출력

버퍼 (Buffer)

데이터를 전송하기 전 일시적으로 보관하는 메모리 임시 저장 공간

- **I/O 비용의 문제:** 디스크에 물리적으로 접근해 1바이트씩 데이터를 직접 쓰고 읽는 연산은 시스템에서 비용이 가장 큼
- **버퍼링의 해결책:** 메모리 버퍼 영역에 데이터를 가득 채웠다가 한 번에 전송함으로써 디스크 접근 횟수를 줄여 성능을 대폭 향상함
- **Files 클래스와 Buffer:** Java NIO의 `Files` 유틸리티를 활용해 간결하고 예외에 안전한 버퍼 입출력 스트림 생성 가능

버퍼를 활용한 파일 텍스트 쓰기

```
1 Path path = Paths.get("data.txt");
2 try (BufferedWriter writer = Files.newBufferedWriter(path)) {
3     writer.write("Java File I/O 학습");
4 } catch (IOException e) {
5     System.out.println("파일 쓰기 실패: " + e.getMessage());
6 }
```

버퍼를 활용한 파일 텍스트 읽기

```
1 Path path = Paths.get("data.txt");
2 try (BufferedReader reader = Files.newBufferedReader(path)) {
3     String line;
4     while ((line = reader.readLine()) != null) {
5         System.out.println(line);
6     }
7 } catch (IOException e) {
8     System.out.println("파일 읽기 실패: " + e.getMessage());
9 }
```

OpenAPI 연동과 바이너리 다운로드

바이너리 HTTP 통신 (Binary HTTP I/O)

텍스트 포맷(JSON, HTML)이 아닌 멀티미디어 바이너리 스트림 데이터를 직접 수신하여 디스크에 저장하는 통신

- **API 결과 분석:** 네이버 이미지 검색 API 등을 통해 수집하고자 하는 이미지 파일의 원본 URL을 획득함
- **HttpClient 다운로드:** 외부 라이브러리 없이 JDK 표준 `HttpClient`를 활용해 대용량 이미지 데이터를 스트림 형태로 직접 전송받음
- **BodyHandlers.ofFile:** HTTP 응답 본문을 즉시 지정된 파일 경로(`Path`)로 스트림 출력하여, 메모리 낭비 없이 파일을 로컬 디스크에 씀

HTTP 통신 기반 이미지 바이너리 저장

```
1 HttpClient client = HttpClient.newHttpClient();
2 HttpRequest request = HttpRequest.newBuilder()
3     .uri(URI.create("https://example.com/image.jpg"))
4     .GET().build();
5
6 Path target = Paths.get("downloaded_image.jpg");
7 client.send(request, HttpResponse.BodyHandlers.ofFile(target));
```